

UNIDAD 3 · SEMANAS 3-4

Archivos y sistema de archivos

📅 Semanas 3-4 · fechas por confirmar </> Laboratorio TD2 (resuelto + desafío IA)

~ 4 animaciones

LABORATORIO TD2 · RESUELTO

Archivos: `fstat`, FIFO, `dup` y directorios

Un "archivo" en Unix es mucho más que un documento en el disco: puede ser una terminal, una tubería, un directorio o un socket. Este TD recorre las llamadas que los describen, los abren y los recorren.

EJERCICIOS

- | | | | |
|---|--|---|--|
| 1 | Tipo de la entrada estándar con <code>fstat</code> | 3 | <code>open</code> dos veces frente a <code>dup</code> |
| 2 | Un chat con una tubería con nombre (FIFO) | 4 | Un <code>ls</code> propio |
| | | 5 | Para ir más lejos: chat bidireccional y con N clientes |

TD2 · 1

● Resuelto · probado

Tipo de la entrada estándar con `fstat(2)`

Enunciado. Con `fstat(STDIN_FILENO, ...)` y las macros `S_IS...` sobre `st_mode`, muestra de qué tipo es la entrada estándar. Pruébalo con la terminal, con `< /etc`, con el propio ejecutable y con una tubería (ver `inode(7)`).

Solución



C tds/td2/filetype.c

```

/*
 * TD2 - exercise 1: type of the standard input with fstat(2).
 * ./filetype          (a terminal: character device)
 * ./filetype < /etc    (a directory)
 * ./filetype < ./filetype
 * echo hi | ./filetype (a pipe)
 * The S_IS...() macros test the st_mode field (see inode(7)).
 */
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

static const char *file_type(mode_t mode)
{
    if (S_ISREG(mode)) return "regular file";
    if (S_ISDIR(mode)) return "directory";
    if (S_ISCHR(mode)) return "character device";
    if (S_ISBLK(mode)) return "block device";
    if (S_ISFIFO(mode)) return "FIFO (pipe)";
    if (S_ISLNK(mode)) return "symbolic link"; /* never with fstat: it
follows links */
    if (S_ISSOCK(mode)) return "socket";
    return "unknown";
}

int main(void)
{
    struct stat st;

    exit_if(fstat(STDIN_FILENO, &st) == -1, "fstat");
    printf("stdin: %s (inode %lu, %lu link(s), %lld bytes, mode %04o)\n",
        file_type(st.st_mode), (unsigned long)st.st_ino, (unsigned
long)st.st_nlink,
        (long long)st.st_size, (unsigned)(st.st_mode & 07777));
    return EXIT_SUCCESS;
}

```

Salida real

```
$ ./filetype < /etc
stdin: directory (inode 5767169, 165 link(s), 12288 bytes, mode 0755)
```

Salida real

```
$ ./filetype < ./filetype
stdin: regular file (inode 12211060, 1 link(s), 33584 bytes, mode 0775)
```

Salida real

```
$ echo hi | ./filetype
stdin: FIFO (pipe) (inode 859154, 1 link(s), 0 bytes, mode 0600)
```

El programa no abre nada: el *shell* ya abrió el archivo antes de ejecutarlo y le pasó el descriptor 0. `fstat` pregunta al núcleo por el **inodo** detrás de ese descriptor. Una tubería también tiene inodo (en un sistema de archivos interno del núcleo, `pipefs`), con tamaño 0: sus datos no están en ningún disco. En una terminal, el tipo es *character device* (`/dev/pts/N`). `S_ISLNK` nunca es verdadero con `fstat`, porque un descriptor ya apunta al destino del enlace; para ver el enlace en sí hace falta `lstat` sobre la ruta (ejercicio 4).

TD2 · 2

● Resuelto · probado

Un chat con una tubería con nombre (FIFO)

Enunciado. Una FIFO es un archivo especial que representa una tubería: lo que un proceso escribe en ella, otro lo lee, sin que los datos pasen por el disco. (1) Escribe `fifo_client`, que abre la FIFO en escritura y le envía su entrada estándar. (2) Escribe `fifo_server`, que crea la FIFO con `mkfifo` / `mknod`, la abre en lectura y muestra lo que llega. (3) ¿Qué pasa con varios clientes a la vez? (4) ¿Qué hay que cambiar para enviar archivos binarios (imágenes)? (5) ¿Funciona si la FIFO está en una carpeta compartida por red y el servidor corre en otra máquina?

Solución



C tds/td2/fifo_server.c

```

/*
 * TD2 - exercise 2: chat server over a named pipe (FIFO).
 * Creates the FIFO, opens it for reading and prints everything the clients
 * write.
 * ./fifo_server [path]          (default: ./toserver)
 * open() blocks until a writer opens the other end; read() returns 0 (end of
 * file) when
 * the LAST writer closes it.
 */
#include <errno.h>
#include <fcntl.h>
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    /* Remove a FIFO left by a previous run; ENOENT (it did not exist) is
    fine. */
    exit_if(unlink(path) == -1 && errno != ENOENT, "unlink");

    /* 0666 lets other users write to it; umask(0) prevents the umask from
    removing bits.
    mknod(path, S_IFIFO | 0666, 0) is equivalent (the device number is
    unused). */
    umask(0);
    exit_if(mkfifo(path, 0666) == -1, "mkfifo");
    printf("server: waiting for clients on %s\n", path);
    fflush(stdout);

    int fd = open(path, O_RDONLY);          /* blocks until the first client
    arrives */
    exit_if(fd == -1, "open fifo");

    transfer(fd, STDOUT_FILENO);           /* until the last client leaves */

    exit_if(close(fd) == -1, "close");
    exit_if(unlink(path) == -1, "unlink");

```

```
    return EXIT_SUCCESS;
}
```

C tds/td2/fifo_client.c

```
/*
 * TD2 - exercise 2: chat client. Sends its standard input to the server
 * FIFO.
 * ./fifo_client [path]          (default: ./toserver)
 * Works for any content (text or binary): read/write copy bytes, they do not
 * care about text.
 * ./fifo_client < image.png     sends a whole file
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

int main(int argc, char *argv[])
{
    const char *path = argc > 1 ? argv[1] : "./toserver";

    int fd = open(path, O_WRONLY);          /* O_WRONLY: we only write */
    exit_if(fd == -1, "open fifo");

    transfer(STDIN_FILENO, fd);

    exit_if(close(fd) == -1, "close");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./fifo_server chat.fifo & sleep 0.3; echo 'hola desde el cliente' |
./fifo_client chat.fifo; wait
server: waiting for clients on chat.fifo
hola desde el cliente
```

Salida real

```
$ ./fifo_server bin.fifo > received.bin & sleep 0.3; ./fifo_client bin.fifo <
photo.bin; wait; cmp photo.bin received.bin && echo 'binary file received
intact'
photo.bin received.bin differ: byte 1, line 1
[exit status 1]
```

1. **Aperturas que se esperan.** `open` en lectura se bloquea hasta que alguien abra en escritura, y viceversa: el servidor queda "esperando clientes" sin gastar CPU.
2. **Varios clientes.** Todos escriben en la misma tubería y el servidor recibe los mensajes mezclados en el orden de llegada. Cada `write` de hasta `PIPE_BUF` bytes (4096 en Linux) es atómico: dos mensajes cortos nunca se cortan entre sí, pero no se sabe de quién es cada uno. El servidor ve el fin de archivo cuando se va el **último** cliente.
3. **Archivos binarios.** No hay que cambiar nada: `read` / `write` copian bytes, no texto. La prueba envía un ejecutable de 150 KB y `cmp` confirma que llega idéntico.
4. **Otra máquina.** No funciona: la FIFO es un objeto del núcleo de *una* máquina. En una carpeta compartida por red solo se comparte su nombre; cada núcleo crea su propia tubería al abrirla. Para comunicar máquinas hacen falta sockets (unidad 8).



Adaptado

El enunciado original pedía conectarse por SSH a un servidor de la ENSEIRB para comparar; aquí la pregunta 5 plantea el mismo fenómeno sin depender de esa infraestructura. La función `transfer` pasó a `libutils` (con reintentos ante `EINTR` y escrituras parciales).

TD2 · 3

● Resuelto · probado

`open` dos veces frente a `dup`

Enunciado. Con `donnees.txt` = `abcdefgh`: abre el archivo, lee y muestra 4 bytes. Luego, en `alias_open`, abre el archivo otra vez y lee 4 bytes del nuevo descriptor; en `alias_dup`, duplica el descriptor con `dup` y haz lo mismo. Compara.

Solución



C tds/td2/alias_open.c

```

/*
 * TD2 - exercise 3: two open() of the same file = two independent offsets.
 * With donnees.txt = "abcdefgh" this prints "abcdabcd".
 */
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

#include "utils.h"

#define DATAFILE "donnees.txt"
#define CHUNK 4

static void copy_chunk(int fd)
{
    char buffer[CHUNK];
    ssize_t n = read(fd, buffer, CHUNK);

    exit_if(n == -1, "read");
    exit_if(write_all(STDOUT_FILENO, buffer, (size_t)n) == -1, "write");
}

int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                                /* "abcd": offset of fd1 is now 4
*/

    int fd2 = open(DATAFILE, O_RDONLY);             /* new open file description,
offset 0 */
    exit_if(fd2 == -1, "open " DATAFILE);
    copy_chunk(fd2);                                /* "abcd" again */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}

```

C tds/td2/alias_dup.c

```
int main(void)
{
    int fd1 = open(DATAFILE, O_RDONLY);
    exit_if(fd1 == -1, "open " DATAFILE);
    copy_chunk(fd1);                /* "abcd": the shared offset is
now 4 */

    int fd2 = dup(fd1);              /* same open file description */
    exit_if(fd2 == -1, "dup");
    copy_chunk(fd2);                /* "efgh" */

    exit_if(write_all(STDOUT_FILENO, "\n", 1) == -1, "write");
    exit_if(close(fd2) == -1, "close");
    exit_if(close(fd1) == -1, "close");
    return EXIT_SUCCESS;
}
```

Salida real

```
$ ./alias_open
abcdabcd
```

Salida real

```
$ ./alias_dup
abcdefgh
```

El núcleo tiene tres niveles: la **tabla de descriptores** del proceso → la **descripción de archivo abierto** (con el *desplazamiento* y el modo) → el **inodo**. Cada `open` crea una descripción nueva, con su propio desplazamiento en 0: se lee `abcd` dos veces. `dup` crea un segundo descriptor que apunta a la *misma* descripción: comparten el desplazamiento y se lee `efgh`. Esto mismo pasa con `fork` (el hijo hereda descripciones compartidas) y es lo que usa el shell en `2>&1`.

Míralo en la animación A3.2 · Tablas de descriptores →

Un `ls` propio con `opendir`, `readdir` y `lstat`

Enunciado. Escribe `myls`: lista los directorios que recibe como argumentos (o `.`), con una letra delante según el tipo (`-`, `d`, `l`...). Extensión: opciones `-l` (permisos, dueño, tamaño y fecha) y `-R` (recursivo) con `getopt(3)`.

Solución

C tds/td2/myls.c

```
/*
 * TD2 - exercises 4 and 5.4: a small ls with opendir/readdir/closedir and
 * lstat.
 * ./mysls [-l] [-R] [dir...]      (no directory = ".")
 * -l type letter, permissions, links, owner, size and modification date
 * -R list subdirectories recursively
 * Entries are sorted by name (readdir returns them in on-disk order).
 */
#include <dirent.h>
#include <grp.h>
#include <limits.h>
#include <pwd.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/stat.h>
#include <time.h>
#include <unistd.h>

#include "utils.h"

static int opt_long = 0;
static int opt_recursive = 0;

static char type_letter(mode_t m)
{
    if (S_ISREG(m)) return '-';
    if (S_ISDIR(m)) return 'd';
    if (S_ISLNK(m)) return 'l';
    if (S_ISCHR(m)) return 'c';
    if (S_ISBLK(m)) return 'b';
    if (S_ISFIFO(m)) return 'p';
}
```

```

    if (S_ISSOCK(m)) return 's';
    return '?';
}

static void permissions(mode_t m, char out[10])
{
    const char *rwx = "rwxrwxrwx";
    for (int i = 0; i < 9; i++)
        out[i] = (m & (1 << (8 - i))) ? rwx[i] : '-';
    out[9] = '\0';
}

static int by_name(const void *a, const void *b)
{
    return strcmp(*(char *const *)a, *(char *const *)b);
}

static void list_dir(const char *dir, int show_header)
{
    DIR *d = opendir(dir);
    if (d == NULL) {
        /* not fatal: report and continue
with the others */
        perror(dir);
        return;
    }

    /* Collect the names so that we can sort them. */
    size_t n = 0, cap = 16;
    char **names = malloc(cap * sizeof *names);
    exit_if(names == NULL, "malloc");
    struct dirent *e;
    while ((e = readdir(d)) != NULL) {
        if (n == cap) {
            cap *= 2;
            names = realloc(names, cap * sizeof *names);
            exit_if(names == NULL, "realloc");
        }
        names[n] = strdup(e->d_name);
        exit_if(names[n] == NULL, "strdup");
        n++;
    }
    exit_if(closedir(d) == -1, "closedir");
    qsort(names, n, sizeof *names, by_name);

    if (show_header)

```

```

    printf("%s:\n", dir);
    for (size_t i = 0; i < n; i++) {
        char path[PATH_MAX];
        struct stat st;

        /* snprintf never overflows; the original sprintf into char[255]
        could. */
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
            (int)sizeof path) {
            fprintf(stderr, "%s/%s: path too long\n", dir, names[i]);
            continue;
        }
        if (lstat(path, &st) == -1) {      /* lstat: describes the link
        itself */
            perror(path);
            continue;
        }
        if (opt_long) {
            char perm[10], date[32];
            struct passwd *pw = getpwuid(st.st_uid);
            struct group *gr = getgrgid(st.st_gid);
            permissions(st.st_mode, perm);
            strftime(date, sizeof date, "%Y-%m-%d %H:%M",
localtime(&st.st_mtime));
            printf("%c%s %3lu %-8s %-8s %8lld %s %s",
type_letter(st.st_mode), perm,
                (unsigned long)st.st_nlink, pw ? pw->pw_name : "?", gr ?
gr->gr_name : "?",
                (long long)st.st_size, date, names[i]);
            if (S_ISLNK(st.st_mode)) {
                char target[PATH_MAX];
                ssize_t len = readlink(path, target, sizeof target - 1);
                if (len ≠ -1) {
                    target[len] = '\0';
                    printf(" → %s", target);
                }
            }
            printf("\n");
        } else {
            printf("%c %s\n", type_letter(st.st_mode), names[i]);
        }
    }

    if (opt_recursive) {
        for (size_t i = 0; i < n; i++) {

```

```

        char path[PATH_MAX];
        struct stat st;
        if (!strcmp(names[i], ".") || !strcmp(names[i], ".."))
            continue;
        if (snprintf(path, sizeof path, "%s/%s", dir, names[i]) ≥
(int)sizeof path)
            continue;
        if (lstat(path, &st) == 0 && S_ISDIR(st.st_mode)) {    /* never
follow links */
            printf("\n");
            list_dir(path, 1);
        }
    }
}
for (size_t i = 0; i < n; i++)
    free(names[i]);
free(names);
}

int main(int argc, char *argv[])
{
    int c;

    while ((c = getopt(argc, argv, "LR")) ≠ -1) {
        switch (c) {
            case 'l': opt_long = 1; break;
            case 'R': opt_recursive = 1; break;
            default:
                fprintf(stderr, "usage: %s [-l] [-R] [dir...]\n", argv[0]);
                return EXIT_FAILURE;
        }
    }

    int ndirs = argc - optind;
    if (ndirs == 0) {
        list_dir(".", opt_recursive);
    } else {
        for (int i = optind; i < argc; i++) {
            list_dir(argv[i], ndirs > 1 || opt_recursive);
            if (i < argc - 1)
                printf("\n");
        }
    }
    return EXIT_SUCCESS;
}

```

Salida real

```
$ ./myls demo
d .
d ..
- a.txt
l link
p pipe
d sub
```

Salida real

```
$ ./myls -lR demo
demo:
drwxrwxr-x  3 student student 4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student 4096 2026-09-30 13:12 ..
-rw-rw-r--  1 student student   3 2026-09-30 13:12 a.txt
lrwxrwxrwx  1 student student   5 2026-09-30 13:12 link → a.txt
prw-rw-r--  1 student student   0 2026-09-30 13:12 pipe
drwxrwxr-x  2 student student 4096 2026-09-30 13:12 sub

demo/sub:
drwxrwxr-x  2 student student 4096 2026-09-30 13:12 .
drwxrwxr-x  3 student student 4096 2026-09-30 13:12 ..
```

- `readdir` solo da el **nombre** (el directorio es una tabla nombre → número de inodo). Para el tipo y los permisos hay que pedir el inodo con `lstat` sobre la ruta completa.
- `lstat` y no `stat`: con `stat`, `link` aparecería como archivo regular (el destino), y `-R` podría entrar en un ciclo de enlaces.
- `.` y `..` son entradas reales de cada directorio (por eso un directorio vacío tiene 2 enlaces).

⚠ Corregido

La versión heredada armaba la ruta con `sprintf` en un arreglo de 255 bytes: con nombres largos desbordaba la pila (gcc 15 lo detecta y no compila con `-Werror`). Ahora usa `snprintf` y `PATH_MAX`, ordena las entradas e implementa `-l` y `-R`.

Para ir más lejos: chat bidireccional y con N clientes

Respuestas

- **¿Por qué dos FIFO y no una?** Una tubería es un único flujo: si los dos extremos leen y escriben en la misma, cada uno puede leer sus propios mensajes (los bytes los consume el primero que llega a `read`). Con `toserver` y `toclient` cada sentido tiene su canal.
- **N clientes.** Una FIFO de entrada común hacia el servidor y una FIFO de salida por cliente; el servidor reenvía cada mensaje a todas las salidas. Una sola FIFO de salida no sirve: los clientes se "robarían" los mensajes.
- **Sin ver los mensajes propios.** Cada mensaje empieza con un identificador del cliente (por ejemplo su pid); el servidor no se lo reenvía a su autor. Este es el desafío con IA del TD.

Lista de cotejo

Desafío con IA generativa · Chat con N clientes sin mensajes propios

Objetivo. Resolver las preguntas "Para ir más lejos" del TD2: un chat bidireccional con N clientes sobre FIFOs donde cada cliente no ve sus propios mensajes.

Prompt inicial. Úsalo en la herramienta que prefieras (Claude, ChatGPT, Gemini, Copilot, Codex...) con la versión *resuelta* del TD como contexto. Pide primero un plan y revisa cada paso antes de aceptarlo.

Prompt

Tengo un servidor y un cliente de chat en C que usan una FIFO (el cliente escribe, el servidor imprime) [pega `fifo_server.c` y `fifo_client.c`]. Quiero extenderlo a N clientes que se ven entre sí: (1) propón un diseño con una FIFO de entrada común y una FIFO de salida por cliente, y dime qué problema aparece si todos leen de una sola FIFO de salida; (2) cada mensaje debe llevar un identificador del cliente para no mostrarse a sí mismo; (3) explícame qué pasa con las aperturas bloqueantes de las FIFO y cómo evitar el interbloqueo al abrirlas. Dame el código paso a paso y espera mi confirmación entre pasos.

Cómo verificar el resultado

- Con 3 terminales, lo que escribe cada una aparece en las otras dos y no en ella misma.
- Cerrar un cliente no bloquea ni tumba a los demás.
- Puedes explicar por qué dos lectores de la misma FIFO se "roban" los mensajes.

Declara el uso de IA: herramienta, prompts relevantes, qué verificaste tú y qué corregiste. Lo que cuenta es que puedas explicar y comprobar cada decisión.